



FUTURE INTERNS · CYBER SECURITY

Read-only assessment of public test APIs
APIs analysed: JSONPlaceholder & ReqRes

API Security Risk Analysis Report

Prepared by Assia EL MOURTADAH
Future Interns — Cyber Security Internship (Task 3)



Table Of Contents

Table Of Contents	2
1. Executive Summary	3
2. Objective and Scope.....	3
2.1. Objective	3
2.2. Scope and Ethics	3
3. Tools Used.....	4
4. Methodology	4
5. APIs Analysed.....	4
5.1. JSONPlaceholder	4
5.2. ReqRes	4
6. Tests Performed	4
7. Identified Risks, Severity and Remediation.....	5
7.1. Severity Scale.....	7
8. Positive Findings.....	8
9. Comparative Analysis: JSONPlaceholder vs ReqRes	8
10. General Recommendations	8
11. Conclusion.....	9
12. References	9

1. Executive Summary

This report presents a read-only API security risk assessment carried out on two public demonstration Application Programming Interfaces (APIs): JSONPlaceholder and ReqRes. The goal is to identify common API security weaknesses, evaluate authentication and access-control mechanisms, and propose clear recommendations, in the same way a security agency would deliver an audit to a SaaS client.

The assessment was conducted strictly ethically: read-only requests (GET) only, on public APIs intended for learning, with no attempt at exploitation, bypass or overload.

The JSONPlaceholder API presents several notable risks, including three of high severity: open endpoints without authentication, excessive exposure of personal data, and a lack of object-level authorization control (BOLA). In contrast, the ReqRes API correctly enforces authentication by rejecting requests without a valid key, which serves as a good-practice reference.

2. Objective and Scope

2.1. Objective

The objective of this assignment is to think like a security consultant in order to:

- ✓ Analyse public or test APIs;
- ✓ Identify the most common API security risks;
- ✓ Assess authentication and access control;
- ✓ Explain risks in plain business language;
- ✓ Suggest clear remediation steps.

2.2. Scope and Ethics

The following activities were allowed and respected: testing public or demo APIs, read-only requests, documentation-based analysis, and inspection of headers, tokens and responses.

The following activities were not allowed and were never performed: any exploitation or bypass attempt, flooding or denial-of-service testing, and attacking private or production APIs.

3. Tools Used

- ✓ Postman sending requests and inspecting responses and headers;
- ✓ Firefox browser and its developer tools (DevTools) reading documentation and observing traffic;
- ✓ Official API documentation (JSONPlaceholder and ReqRes);
- ✓ Word processor (Microsoft Word) for writing this report.

4. Methodology

The approach followed the recommended steps for an API risk audit:

- ✓ Selection of the test APIs (JSONPlaceholder and ReqRes);
- ✓ Reading the documentation of each API;
- ✓ Testing the endpoints with Postman;
- ✓ Inspecting authentication requirements, headers and response data;
- ✓ Identifying security risks;
- ✓ Classifying the severity of each risk (Low / Medium / High);
- ✓ Proposing remediation steps;
- ✓ Documenting all findings.

5. APIs Analysed

5.1. JSONPlaceholder

A public test API that simulates a database of users and posts. It requires no authentication, which makes it ideal for illustrating data-exposure risks.

5.2. ReqRes

An API testing platform that offers authentication mechanisms (API keys and session tokens). During testing it returned a 401 error when no valid key was supplied, demonstrating that authentication is effectively enforced.

6. Tests Performed

The table below summarises the requests performed together with the corresponding observations:

Test performed	Endpoint	Method	Code	Observation
List of users	/users	GET	200	Data exposed without authentication
Specific user	/users/1	GET	200	Direct object access
Another user (modified ID)	/users/2	GET	200	Access to another user's data (BOLA)
Non-existent ID	/users/9999	GET	404	Error handled correctly
Invalid input (text)	/users/abc	GET	404	Input validation works
Request without API key	reqres.in/api/users	GET	401	Authentication enforced (good practice)

7. Identified Risks, Severity and Remediation

The following register lists the detected risks, classified by severity, with their business impact and the recommended remediation measures:

No.	Risk	Affected API	Severity	Business impact	Remediation
1	Open / unauthenticated endpoint	JSONPlaceholder	High	Anyone can retrieve the entire user base, leading to a mass data	Require authentication (API key or token) on all sensitive endpoints.

No.	Risk	Affected API	Severity	Business impact	Remediation
				leak and GDPR exposure.	
2	Excessive data exposure	JSONPlaceholder	High	Email, phone, address and GPS coordinates can be abused for phishing, spam or geolocation.	Return only strictly necessary fields (data minimisation).
3	Broken Object Level Authorization (BOLA / IDOR)	JSONPlaceholder	High	Other users' data can be accessed simply by changing the identifier in the URL.	Enforce server-side checks that the user is allowed to access the requested object.
4	Permissive CORS configuration	JSONPlaceholder	Medium	A header allows credential requests from broad origins, creating a cross-origin abuse risk.	Restrict allowed origins to an explicit list of trusted domains.

No.	Risk	Affected API	Severity	Business impact	Remediation
5	Missing security headers	JSONPlaceholder	Low	Absence of HSTS, X-Frame-Options and CSP exposes the service to clickjacking and HTTPS downgrade.	Add HSTS, X-Frame-Options and Content-Security-Policy headers server-side.
6	Technical information disclosure	JSONPlaceholder	Low	The X-Powered-By header reveals the technology stack (Express), easing the targeting of known exploits.	Remove or hide the X-Powered-By header.

7.1. Severity Scale

High: sensitive data exposed and easy to exploit; immediate legal or reputational impact.

Medium: a real but conditional risk, or a more limited impact.

Low: a missing good practice with low direct impact.

8. Positive Findings

The analysis is not limited to weaknesses: it also records the good practices observed, which serve as reference points.

Good practice observed	API	Observed evidence
Rate limiting is active	JSONPlaceholder	Header x-ratelimit-limit: 1000
Protection against MIME sniffing	JSONPlaceholder	Header x-content-type-options: nosniff
Authentication correctly enforced	ReqRes	401 response without a valid API key
Correct handling of invalid input	JSONPlaceholder	404 for /users/9999 and /users/abc

9. Comparative Analysis: JSONPlaceholder vs ReqRes

The comparison highlights two opposite security postures. JSONPlaceholder favours ease of use: its endpoints are completely open, which makes learning easy but exposes data widely. ReqRes, on the other hand, requires an API key and rejects unauthenticated requests, illustrating an approach more consistent with the requirements of a production API.

This comparison shows that, beyond detecting flaws, an analyst must also recognise protection mechanisms that are correctly implemented.

10. General Recommendations

- ✓ Require robust authentication (API key or token) on all endpoints exposing data;
- ✓ Enforce server-side authorization to prevent access to other users' objects (BOLA prevention);
- ✓ Limit returned fields to the strict minimum (data minimisation);
- ✓ Restrict the CORS policy to trusted origins;
- ✓ Add security headers (HSTS, X-Frame-Options, Content-Security-Policy);

- ✓ Hide headers that reveal the technology stack (X-Powered-By);
- ✓ Maintain the rate limiting already in place to prevent abuse.

11. Conclusion

This assessment identified six security risks on the JSONPlaceholder API, three of them of high severity, related to the absence of authentication, excessive data exposure and the lack of access control. It also highlighted good practices, in particular rate limiting and ReqRes's strict enforcement of authentication.

The proposed recommendations, simple and concrete, would significantly reduce the attack surface while preserving the functionality of the services. This work illustrates the approach of a modern API security audit, focused on identifying risk rather than exploiting it.

12. References

- ✓ OWASP API Security Top 10 official OWASP project;
- ✓ API Security Checklist (shieldfy) practical reference;
- ✓ JSONPlaceholder and ReqRes documentation.